Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)

Факультет прикладной информатики
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии

О Т Ч Е Т
по лабораторной работе № 3

Дисциплина: Метод множителей Лагранжа
Тема: «Метод ломаных»

Обучающийся: Данилова Айаана Васильевна, группа K3341, поток 1.4
Преподаватель: Микрюкова А.С.

Санкт-Петербург 2026

ЦЕЛЬ РАБОТЫ

Решение задачи нахождения экстремума методом множителей Лагранжа с последующей реализацией на языке Python и сверка получившихся результатов.

Вариант работы: 10

10. Найти экстремумы функции

$$f(x, y, z) = x + 2y + z$$

при условиях

$$\begin{cases} x^2 + y^2 + z^2 = 22 \\ x = y \end{cases}$$

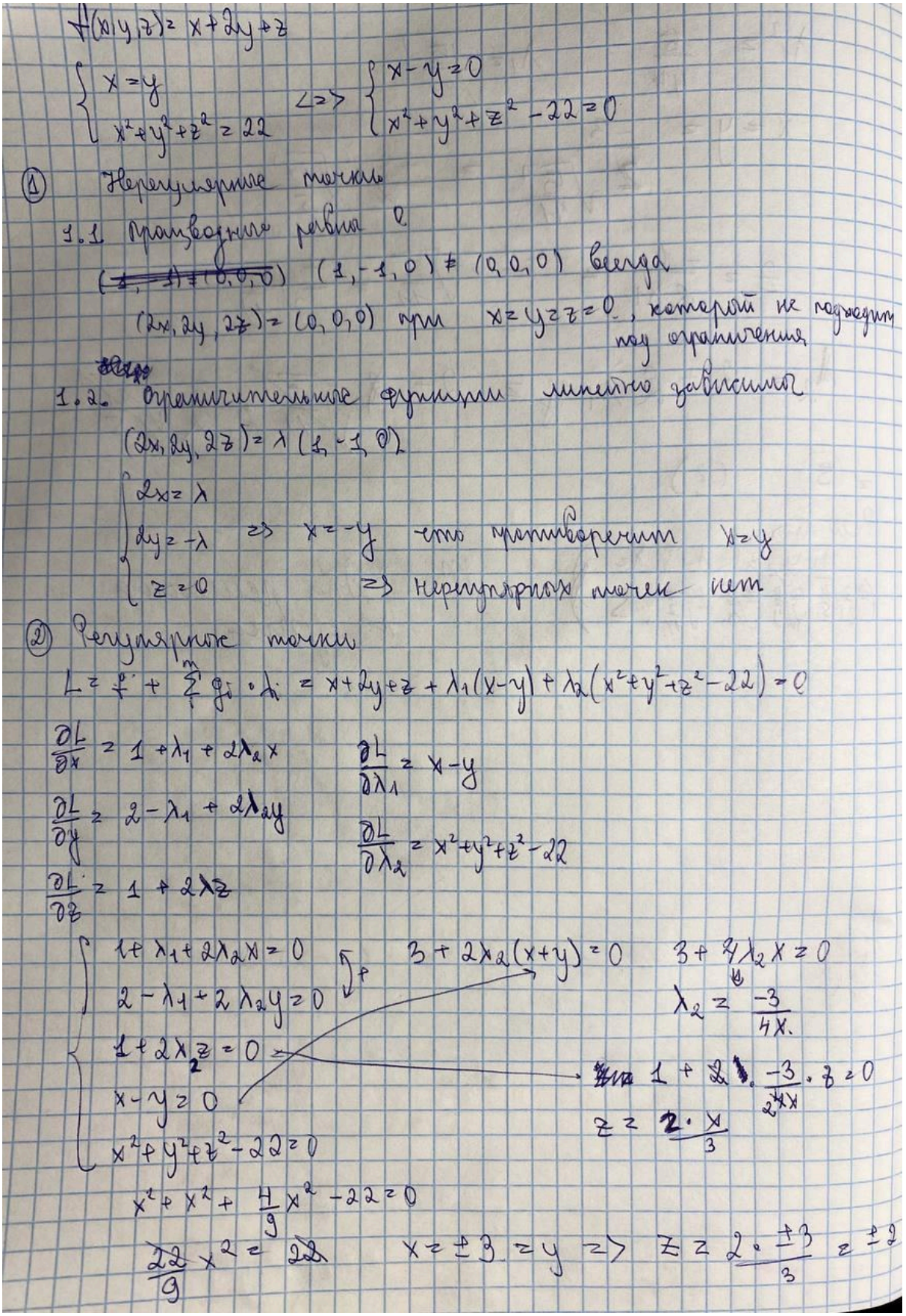
ХОД РАБОТЫ

В ходе выполнения работы была решена задача поиска условного экстремума функции нескольких переменных с ограничениями типа равенств. Для выявления нерегулярных точек, в которых нарушается условие регулярности ограничений (то есть матрица Якоби системы функций ограничений имеет неполный ранг), в программном коде использовалось условие линейной зависимости строк матрицы градиентов ограничений. Это условие эквивалентно равенству нулю всех миноров максимального порядка, составленных из частных производных функций ограничений по всем независимым переменным. Данные уравнения, определяющие неполный ранг матрицы Якоби, объединены с исходной системой уравнений ограничений, приведенных к нулевой правой части, в общую систему для поиска всех нерегулярных критических точек. Для вычисления частных производных, формирования системы уравнений и ее аналитического решения использовалась библиотека SymPy со следующими функциями:

- sp.sympify(a) - преобразует строку или объект в математическое выражение SymPy
- sp.Eq(lhs, rhs) - создает математическое уравнение вида lhs = rhs
- sp.symbols(names) - создает символьные переменные для математических вычислений
- sp.Matrix(elements) - создает матрицу из переданных элементов или выражений

- sp.S(value) - преобразует значение в соответствующий тип SymPy (например, число 0)
- sp.solve(system, symbols) - решает систему уравнений относительно заданных переменных
- sp.diff(f, x) - вычисляет частную производную функции f по переменной x

Письменное решение:



x, y, z

Тогда критические точки: (3, 3, 2)
(-3, -3, -2)

③ Проверка

$3 = 3$ ✓
$-3 = -3$

$9 + 9 + 4 - 22 = 0$ ✓

④ Определение характера найденных точек

$f(3, 3, 2) = 3 + 6 + 2 = 14 \Rightarrow$ глобальный максимум

$f(-3, -3, -2) = -14 \Rightarrow$ глобальный минимум

⑤ Определение оптимального значения целевой функции

Т.к была поставлена задача нахождения экстремумов и было найдено только 2 значения, то они соответственно и будут являться глобальными минимумом и максимумом

Отв: (3, 3, 2) – макс. экстремум

(-3, -3, -2) – мин. экстремум

Листинг кода:

```
import itertools
import numpy as np
import sympy as sp

def parse_data(data: dict):
  sym_naming = ['x', 'y', 'z']
  n: int = data['n']
  f_expr = sp.sympify(data['f'])

  m = data['m']
  if len(data['g']) != m:
    print("[ERROR]: Incorrect count of limiting functions")
    return

  g_exprs = []
  for i in range(m):
    left, right = data['g'][i].split('=')
    expr_left = sp.sympify(left)
    expr_right = sp.sympify(right)

    eq = sp.Eq(expr_left, expr_right)
    eq = eq.lhs - eq.rhs

    g_exprs.append(eq)

  return n, f_expr, m, g_exprs


def find_nonreg_solves(n: int, f_expr: sp.sympify, m: int, g_exprs:
list[sp.sympify]):
  sym_naming = ['x', 'y', 'z']
  syms = sp.symbols(sym_naming[:n])
  # матрица со всеми частными производными длины m
  J = sp.Matrix(g_exprs).jacobian(syms)

  # условие линейной зависимости - все миноры m x m равны нулю
  minor_exprs = []
  if m < n:
    for cols in itertools.combinations(range(n), m):
      minor_exprs.append(J[:, cols].det())
  else:
    minor_exprs.append(sp.S(0))

  system = g_exprs + minor_exprs
  nonreg_points = sp.solve(system, syms, dict=True)
```

```
  results = []
  for point in nonreg_points:
    f_val = f_expr.subs(point)
    results.append({
        'point': point,
        'f_val': f_val,
        'type': 'non-regular'
    })

  return results


def find_reg_solves(n:int, f_expr:sp.sympify, m: int, g_exprs:
list[sp.sympify]):
  L = f_expr
  lambdas_naming = ['k1', 'k2', 'k3']
  sym_naming = ['x', 'y', 'z']
  syms = sp.symbols(sym_naming[:n])

  lambdas = sp.symbols(lambdas_naming[:m])
  for i in range(m):
    L += g_exprs[i] * lambdas[i]

  diffs = []
  for i in syms + lambdas:
    diffs.append(sp.diff(L, i))

  reg_points = sp.solve(diffs, syms + lambdas, dict=True)
  results = []
  for point in reg_points:
    f_val = f_expr.subs(point)
    results.append({
        'point': point,
        'f_val': f_val,
        'type': 'regular'
    })

  return results


def recognize_type(results: list[dict]):
    valid_points = []
    for p in results:
        try:
            val = float(p['f_val'])
            valid_points.append({'point': p['point'], 'f_val': val, 'type':
p['type']})
        except TypeError:
```

```
            pass

    if not valid_points:
        print("[-] Действительных точек не найдено.")
        return

    valid_points.sort(key=lambda item: item['f_val'], reverse=True)

    global_max = valid_points[0]
    global_min = valid_points[-1]

    # оставляем только те, строковое представление которых не начинается на
'k'
    def format_coords(point_dict):
        filtered = [f"{k} = {v}" for k, v in point_dict.items() if not
str(k).startswith('k')]
        return ", ".join(filtered)

    print(f"[МАКСИМУМ] f = {global_max['f_val']}")
    print(f"   Координаты: {format_coords(global_max['point'])}")
    print(f"   Природа:    {global_max['type']} точка\n")

    print(f"[МИНИМУМ]  f = {global_min['f_val']}")
    print(f"   Координаты: {format_coords(global_min['point'])}")
    print(f"   Природа:    {global_min['type']} точка\n")

    print("="*45)
    print("ПОЛНЫЙ СПИСОК КАНДИДАТОВ")
    print("="*45)

    for i, data in enumerate(valid_points, 1):
        f_val = data['f_val']
        coords_str = format_coords(data['point'])
        p_type = data['type']

        if data == global_max:
            status = "Глобальный максимум"
        elif data == global_min:
            status = "Глобальный минимум"
        else:
            status = "Стационарная (локальный экстр. или седло)"

        print(f"{i}. {status}")
        print(f"   f = {f_val} | Точка: ({coords_str}) | Тип: {p_type}\n")


def main():
  n, f, m, gs = parse_data(input)
```

```
  nonreg = find_nonreg_solves(n, f, m, gs)
  reg = find_reg_solves(n, f, m, gs)
  recognize_type(nonreg + reg)

main()
```

ВЫВОДЫ

В процессе работы было очень полезно самостоятельно проектировать логику работы с готовыми функциями библиотеки и заново применять фундаментальные знания, полученные на прошлых курсах. Такой подход отлично развивает инженерное мышление и помогает лучше понять внутреннее устройство математических алгоритмов оптимизации. Для моей профессиональной специфики DevOps-инженера подобные методы поиска экстремумов можно успешно применять в задачах автоматического масштабирования инфраструктуры. Например, они помогают находить оптимальное соотношение ресурсов процессора и памяти для сервисов, минимизируя задержки ответов и затраты на облачные вычисления. Кроме того, этот математический аппарат полезен при настройке порогов срабатывания систем мониторинга для предотвращения аварийных ситуаций.

ВИДЕО-РЕФЛЕКСИЯ

Ссылка на видео-рефлексию: 3.MOV